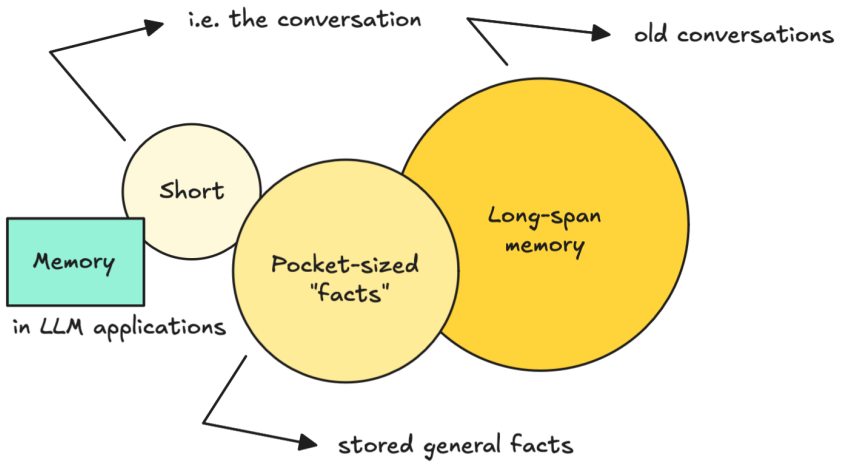




AI ARCHITECTURE

Memory for RAG & Agents

How AI systems remember,
retrieve, and reason like a
human brain.



AI Coach John
FOLLOW



PROITBRIDGE
Solve For Better Future

↻ Repost

What is Memory & Why It Matters

Without memory, your AI is goldfish-brained.

- ✗ Every call starts from zero. No context. No history.
- ⚠ In RAG: retrieves docs but forgets what was discussed 2 turns ago.
- ⚠ In Agents: repeats subtasks, loses user history, breaks flows.

REAL SCENARIO

User: "What about my last order?"

Bot: "I'm sorry, I don't have context about previous conversations."

That's a memoryless agent. Frustrating. Useless.

4 TYPES OF MEMORY

-  **Sensory:** raw current input (your prompt tokens)
-  **Short-Term:** active context window, session-based
-  **Long-Term:** persisted storage across sessions (Vector DB)
-  **Episodic:** structured logs of past sessions with timestamps



In-Context Memory (Short-Term)

What it is: Everything inside the active LLM context window.

SCENARIO: CODING ASSISTANT

Turn 1: "Write a Python function to parse JSON"

Turn 4: "Now add error handling to that"

Without in-context memory, Turn 4 fails. The model has no idea what "that" refers to.

⚙️ HOW TO IMPLEMENT:

- ✓ Append full conversation history to every prompt
- ✓ Use sliding window to drop oldest turns when context fills

```
history = [  
    {"role": "user", "content": "Write a Python function..."},  
    {"role": "assistant", "content": "Here's the function..."},  
]  
response = client.chat(messages=history + [new_message])
```

Frameworks: LangChain ConversationBufferMemory, LlamaIndex ChatMemoryBuffer

Limit: Context window size works great until it doesn't



AI Coach John
FOLLOW



 Repost

Vector / Semantic Memory (Long-Term)

What it is: Past conversations/facts stored as embeddings retrieved by similarity.

SCENARIO: PERSONAL FINANCE BOT

Month 1: You told it your salary, goals, and risk appetite

Month 3: You ask "Should I invest in index funds?"

Without it: asks for your profile again. Every. Single. Time.

With it: retrieves your past context and answers in 10 seconds.

</> HOW TO IMPLEMENT:

```
from langchain.memory import VectorStoreRetrieverMemory
from langchain.vectorstores import FAISS

vectorstore = FAISS.from_texts(past_memories, embeddings)
memory = VectorStoreRetrieverMemory(
    retriever=vectorstore.as_retriever()
)
```

Frameworks: LangChain, Mem0, Chroma, Pinecone, Weaviate

Best for: Long-running assistants, user personalization



AI Coach John
FOLLOW



Repost

Episodic Memory

What it is: Structured logs of past sessions with timestamps and metadata.

SCENARIO: AI RECRUITER

June session: Screened 40 candidates for backend role, shortlisted 5

August session: "Find candidates similar to who we shortlisted last time"

Without it: you start from scratch.

With it: agent pulls the June episode and reasons on top of it.

</> HOW TO IMPLEMENT:

```
episode = {  
  "session_id": "abc123",  
  "timestamp": "2025-06-01",  
  "summary": "Screened backend candidates, shortlisted 5",  
  "tags": ["hiring", "backend", "shortlist"]  
}  
# Store in DB retrieve on next relevant session
```

Frameworks: Mem0, Zep, PostgreSQL + pgvector

Best for: Personal AI assistants, multi-session agents



AI Coach John
FOLLOW



 **Repost**

External / Tool Memory

What it is: Agent reads/writes memory via tools autonomously (files, DBs, APIs).

SCENARIO: AUTONOMOUS RESEARCHER

Part 3: Researches LLM benchmarks and saves key findings

Part 7: Needs to cross-reference what it found in Part 3

Without tool memory: the agent has no way to look back.

With it: agent calls `search_memory("LLM benchmarks")` and retrieves instantly.

</> HOW TO IMPLEMENT:

```
tools = [  
    Tool(name="SaveMemory", func=memory_store.save),  
    Tool(name="SearchMemory", func=memory_store.search),  
]  
agent = initialize_agent(tools, llm, agent="zero-shot-react")
```

Frameworks: LangGraph, AutoGen, CrewAI, LangChain Agents

Best for: Autonomous agents running long multi-step tasks



AI Coach John
FOLLOW



 **Repost**

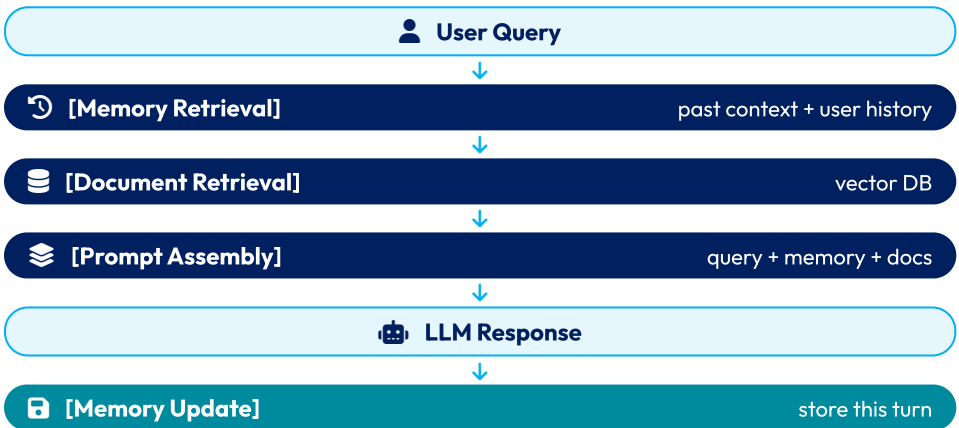
Memory in RAG Pipeline

🔗 SCENARIO: LEGAL ASSISTANT

Query 1: "Explain Section 80C of Indian Income Tax Act"

Query 2: "How does it compare to what you explained earlier about 80D?"

Memory-augmented RAG retrieves docs + injects past context into prompt.



💡 **Memory makes RAG stateful it knows what was already asked and answered.**



AI Coach John
FOLLOW



↺️ **Repost**

Frameworks Cheatsheet

A quick guide to choosing the right tool for your memory needs.

Need	Tool
Simple chat history	LangChain ConversationBufferMemory
Summarized history	LangChain ConversationSummaryMemory
Semantic memory	MemO, Zep, FAISS + LangChain
Agent memory	LangGraph, CrewAI, AutoGen
Production memory	Pinecone + MemO + PostgreSQL



Ready to Build Stateful Agents?

Start small with context windows, then scale to Vector DBs as complexity grows.



AI Coach John
FOLLOW



PROITBRIDGE
Solve For Better Future

 Repost